

Guía Completa: backup-system.sh

¿Por qué rsync y no tar?

Criterio	rsync	tar
Tipo de backup	Incremental por defecto	Completo (normalmente)
Velocidad	Solo transfiere cambios	Siempre todo el contenido
Espacio en disco	Muy eficiente	Crece rápido
Restauración	Directa (copias normales)	Requiere descomprimir
Navegación	Explorador de archivos normal	Requiere herramientas
Redes lentas	Ideal	Ineficiente
Integridad parcial	Puede continuar si se interrumpe	Backup corrupto si falla

Conclusión: `rsync` es la mejor opción para backups semanales al disco externo. Los archivos en destino son copias reales navegables, la transferencia solo mueve lo que cambió, y la restauración es tan simple como copiar.

1. Explicación por secciones importantes

```
set -euo pipefail
```

```
bash
```

```
set -euo pipefail
```

- `-e` → El script para ante cualquier error (no continúa silenciosamente).
- `-u` → Error si se usa una variable no definida (evita bugs silenciosos).
- `-o pipefail` → Un pipe falla si cualquier comando en la cadena falla (ej: `cmd1 | cmd2`).

Variables configurables

```
bash
```

```
DISK_UUID="xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx"  
MOUNT_POINT="/mnt/backups"  
MAX_BACKUPS=4  
MIN_FREE_SPACE_MB=5120
```

Todas las decisiones de configuración están al principio. El resto del script nunca necesita ser modificado.

Sistema de logs con colores

```
bash

log_info()  → verde    → mensajes informativos normales
log_warn()  → amarillo → advertencias no fatales
log_error() → rojo     → errores (también va a stderr con >&2)
log_step()  → azul     → cabeceras de etapa
log_done()  → cian     → confirmaciones de éxito
```

Cada función escribe simultáneamente en terminal (con color) y en `/var/log/backup-script.log` (sin códigos ANSI).

Trap de limpieza

```
bash

trap cleanup EXIT
```

La función `cleanup` se ejecuta **siempre** al terminar el script, ya sea por éxito, error o señal externa (Ctrl+C). Garantiza que el disco externo siempre quede desmontado.

Detección del disco por UUID

```
bash

device="$(blkid --uuid "${DISK_UUID}" 2>/dev/null || true)"
```

`blkid` busca el dispositivo (`/dev/sdb1`, etc.) a partir del UUID. Usar UUID es más seguro que `/dev/sdX` porque la letra puede cambiar entre reinicios o si conectas otros discos.

Verificación de espacio

```
bash

free_mb="$(df --block-size=1M --output=avail "${MOUNT_POINT}" | tail -1 | tr -d ' ')"
```

Extrae el espacio libre en MB con `df`, eliminando espacios extra. Si no hay suficiente espacio, el script aborta antes de empezar.

Política de retención

```
bash
```

```
mapfile -t backups < <(find "${BACKUP_ROOT}" -maxdepth 1 -mindepth 1 -type d | sort)
```

Lista todos los subdirectorios de backup (uno por fecha) ordenados alfabéticamente. Como el formato es `YYYY-MM-DD`, el orden alfabético = orden cronológico. Elimina los más antiguos hasta que quede espacio para el nuevo.

El comando rsync

```
bash
```

```
rsync -aAXv --delete --numeric-ids --human-readable --stats \  
    --exclude=... \  
    /home/ \  
    /mnt/backups/backups/2026-05-10/home/
```

- `-a` → Archivo: preserva permisos, propietarios, timestamps, links simbólicos, recursivo.
 - `-A` → Preserva ACLs (listas de control de acceso).
 - `-X` → Preserva atributos extendidos (xattrs).
 - `--delete` → Elimina en destino lo que ya no existe en origen (sincronización real).
 - `--numeric-ids` → No traduce UIDs/GIDs por nombre (seguro entre sistemas distintos).
 - La barra final en `/home/` copia el **contenido** de `/home`, no la carpeta en sí.
-

2. Configuración inicial

Paso 1: Obtener el UUID del disco externo

```
bash
```

```
# Conecta el disco, luego:
```

```
sudo blkid
```

```
# Salida ejemplo:
```

```
# /dev/sdb1: UUID="a1b2c3d4-e5f6-7890-abcd-ef1234567890" TYPE="ext4"
```

```
# O de forma más limpia:
```

```
sudo blkid -o list
```

Paso 2: Editar el script

```
bash
```

```
sudo nano /usr/local/sbin/backup-system.sh
```

Cambia estas líneas al principio:

```
bash
```

```
DISK_UUID="a1b2c3d4-e5f6-7890-abcd-ef1234567890" # Tu UUID real
```

```
MIN_FREE_SPACE_MB=10240 # Ajusta según tus necesidades (10 GB)
```

```
BACKUP_SOURCES=(
```

```
    "/home"
```

```
    "/etc"
```

```
    "/var"
```

```
)
```

```
EXTRA_SOURCES=(
```

```
    "/opt" # Descomenta si quieres incluir /opt
```

```
    # "/root" # Descomenta si quieres el home de root
```

```
)
```

Paso 3: Instalar el script

```
bash
```

```
# Copiar al directorio de scripts del sistema
```

```
sudo cp backup-system.sh /usr/local/sbin/backup-system.sh
```

```
# Dar permisos de ejecución solo para root
```

```
sudo chmod 700 /usr/local/sbin/backup-system.sh
```

```
sudo chown root:root /usr/local/sbin/backup-system.sh
```

Paso 4: Prueba manual

```
bash
```

```
# Siempre prueba antes de automatizar
```

```
sudo /usr/local/sbin/backup-system.sh
```

```
# Revisa el log inmediatamente después
```

```
sudo tail -100 /var/log/backup-script.log
```

3. Configuración de cron semanal

Opción A: crontab del sistema (recomendado)

```
bash
```

```
sudo crontab -e
```

Añade esta línea al final:

```
cron
```

```
# Backup completo cada domingo a las 02:00 AM
0 2 * * 0 /usr/local/sbin/backup-system.sh >> /var/log/backup-cron.log 2>&1
```

Explicación de la expresión cron:

```
0    → minuto 0
2    → hora 2 (02:00 AM)
*    → cualquier día del mes
*    → cualquier mes
0    → día de semana 0 = domingo (0 o 7)
```

Opción B: archivo en /etc/cron.d/ (más organizado)

```
bash
```

```
sudo nano /etc/cron.d/backup-system
```

Contenido:

```
cron
```

```
# Backup semanal del sistema - cada domingo a las 02:00
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

0 2 * * 0 root /usr/local/sbin/backup-system.sh >> /var/log/backup-cron.log 2>&1
```

Opción C: systemd timer (más moderno y flexible)

```
bash
```

```
# Crear el servicio
sudo nano /etc/systemd/system/backup-system.service
```

ini

```
[Unit]
Description=Backup semanal del sistema
After=network.target

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/backup-system.sh
User=root
StandardOutput=append:/var/log/backup-cron.log
StandardError=append:/var/log/backup-cron.log
```

bash

```
# Crear el timer
sudo nano /etc/systemd/system/backup-system.timer
```

ini

```
[Unit]
Description=Ejecutar backup-system cada domingo

[Timer]
OnCalendar=Sun *-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

bash

```
# Activar el timer
sudo systemctl daemon-reload
sudo systemctl enable --now backup-system.timer

# Verificar
sudo systemctl list-timers backup-system.timer
```

4. Restauración de backups

Ver qué backups existen

```
bash
```

```
# Monta el disco manualmente
```

```
sudo mount UUID="tu-uuid-aqui" /mnt/backups
```

```
# Lista los backups disponibles
```

```
ls -lh /mnt/backups/backups/
```

```
# Salida ejemplo:
```

```
# drwxr-xr-x 2026-04-14
```

```
# drwxr-xr-x 2026-04-21
```

```
# drwxr-xr-x 2026-04-28
```

```
# drwxr-xr-x 2026-05-05
```

Restaurar un archivo o directorio específico

```
bash
```

```
# Restaurar un archivo concreto
```

```
sudo cp /mnt/backups/backups/2026-05-05/home/usuario/.bashrc /home/usuario/.bashrc
```

```
# Restaurar un directorio completo
```

```
sudo rsync -aXv /mnt/backups/backups/2026-05-05/home/usuario/ /home/usuario/
```

```
# Restaurar /etc completo (!cuidado! sobrescribe configuración actual)
```

```
sudo rsync -aXv --dry-run /mnt/backups/backups/2026-05-05/etc/ /etc/
```

```
# Quita --dry-run cuando estés seguro
```

Restauración completa del sistema (desde live USB)

```
bash
```

```
# 1. Arranca desde un live USB con Linux
```

```
# 2. Monta tu partición raíz
```

```
sudo mount /dev/sdaX /mnt/sistema
```

```
# 3. Monta el disco de backup
```

```
sudo mount UUID="tu-uuid" /mnt/backups
```

```
# 4. Restaura cada directorio
```

```
sudo rsync -aXv /mnt/backups/backups/2026-05-05/home/ /mnt/sistema/home/
```

```
sudo rsync -aXv /mnt/backups/backups/2026-05-05/etc/ /mnt/sistema/etc/
```

```
sudo rsync -aXv /mnt/backups/backups/2026-05-05/var/ /mnt/sistema/var/
```

5. Recomendaciones de seguridad

Permisos del script

```
bash
```

```
# Solo root puede leer y ejecutar
```

```
sudo chmod 700 /usr/local/sbin/backup-system.sh
```

```
sudo chown root:root /usr/local/sbin/backup-system.sh
```

Permisos del log

```
bash
```

```
# Solo root puede leer el log (puede contener info sensible)
```

```
sudo chmod 640 /var/log/backup-script.log
```

```
sudo chown root:adm /var/log/backup-script.log
```

Cifrado del disco externo (muy recomendado)

```
bash
```

```
# Formatea el disco con LUKS (cifrado)
```

```
sudo cryptsetup luksFormat /dev/sdb1
```

```
# Monta el disco cifrado
```

```
sudo cryptsetup luksOpen /dev/sdb1 backup_disk
```

```
sudo mount /dev/mapper/backup_disk /mnt/backups
```

```
# Para automatizar el montaje cifrado, guarda la clave en un archivo protegido:
```

```
sudo cryptsetup luksAddKey /dev/sdb1 /root/.backup_keyfile
```

```
sudo chmod 400 /root/.backup_keyfile
```

```
# Luego en el script, ajusta la línea de montaje:
```

```
# cryptsetup luksOpen --key-file /root/.backup_keyfile /dev/sdb1 backup_disk
```

```
# mount /dev/mapper/backup_disk "${MOUNT_POINT}"
```

Verificar integridad del backup

```
bash
```

```
# Genera checksums para verificar integridad posterior
```

```
sudo find /mnt/backups/backups/2026-05-05 -type f -exec sha256sum {} \; \
    > /mnt/backups/checksums-2026-05-05.txt
```

```
# Verificar más tarde
```

```
sudo sha256sum --check /mnt/backups/checksums-2026-05-05.txt
```

Logrotate para el log del backup


```
bash
```

```
sudo nano /etc/logrotate.d/backup-system
```

```
/var/log/backup-script.log {  
    weekly  
    rotate 12  
    compress  
    delaycompress  
    missingok  
    notifempty  
}
```

6. Solución de problemas comunes

Problema	Causa probable	Solución
UUID no encontrado	Disco no conectado o UUID incorrecto	Conecta el disco y verifica con <code>blkid</code>
Espacio insuficiente	Disco lleno	Baja <code>MAX_BACKUPS</code> o aumenta el disco
<code>rsync: permission denied</code>	Script no ejecutado como root	Usa <code>sudo</code> o configura cron como root
El disco ya está montado	Backup anterior no terminó bien	<code>sudo umount /mnt/backups</code> manualmente
<code>set -e</code> interrumpió <code>rsync</code>	Error en un directorio fuente	Revisa el log para el directorio específico